

MarcoSmiles

Human-Computer Interaction & Music: A Machine Learning approach for music performances

Progettazione Sistema

In fase di progettazione, abbiamo diviso il sistema in 3 moduli:

1. **Modulo Unity:** interfacciamento Leap Motion con Unity3D
2. **Modulo Machine Learning:** sviluppo ed utilizzo di un machine learning classifier
3. **Modulo Sonoro:** generazione del suono in real-time

Modulo Machine Learning:

Registrazione dati, Training & Testing

Artificial Neural Network



Cos'è una Artificial Neural Network?

Ispirata all'insieme di neuroni del cervello umano, è a sua volta un insieme di interconnessioni di informazioni o “neuroni artificiali”.

Come funzionano i neuroni artificiali?

Ogni neurone fa uso di una cosiddetta *funzione di attivazione*:

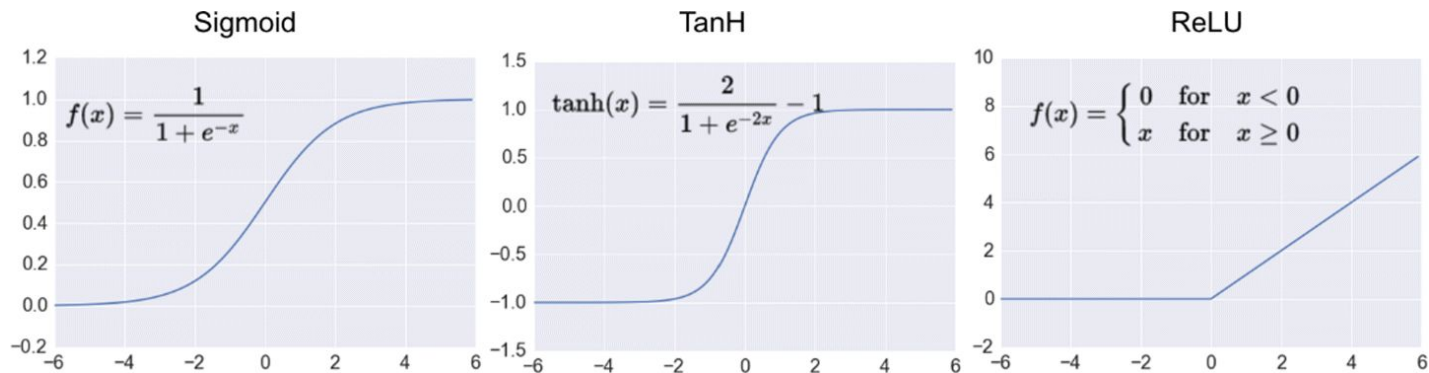
$$y = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

Dove w_i è la matrice dei pesi (**weights**) assegnati a ciascun input e b (**bias**) è un valore di “soglia” per il peso relativo.

La rete neurale combina i valori ottenuti dai risultati della *funzione di attivazione* per ogni neurone.

Ottimizzazione

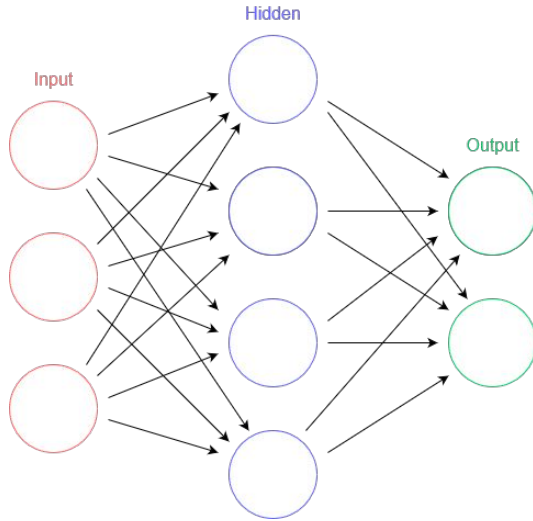
Per “filtrare” le informazioni che vogliamo far utilizzare dalla rete neurale, possiamo fare uso delle seguenti funzioni di attivazione, che hanno il lavoro di regolare i valori ricevuti in ingresso dalla rete.



La funzione da noi selezionata è la **ReLU** avendo restituito risultati più soddisfacenti rispetto le altre due funzioni provate.

Artificial Neural Network

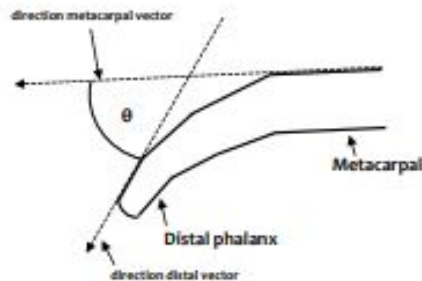
Struttura di una Rete Neurale



- **Input Layer:** Riceve e prepara i segnali ricevuti inviandoli poi all'Hidden Layer;
- **Hidden Layer:** Elabora tutti i dati ricevuti dall'Input Layer;
- **Output Layer:** Ultimo livello, dove vengono raccolti tutti i risultati dell'Hidden Layer e combinati per restituire la previsione;

LeapMotion Controller

Abbiamo utilizzato un *LeapMotion Controller* per ottenere determinate informazioni sulle posizioni delle mani e delle dita, *chiamate features*.



Finger Flexion (FF)



Nearest Finger Angles (NFA)

Queste informazioni verranno utilizzate dal classificatore per realizzare l'associazione "nota musicale - posizione mani"

Paradigmi di apprendimento

Vi sono 3 principali paradigmi di apprendimento:

- **Apprendimento Supervisionato:** necessita dati di input e output atteso;
- **Apprendimento Non Supervisionato:** necessita soltanto dati di input, provvede da solo a stimare un output;
- **Apprendimento per Rinforzo:** metodo basato sulle ricompense ed opera come un automa finito deterministico;

Essendo le informazioni in nostro possesso sufficienti a soddisfare i requisiti per **l'apprendimento supervisionato**, abbiamo adottato questa metodologia.

Ipotesi Implementative

Sono state studiate diverse librerie:

- **Unity ML-Agents:** Libreria Ufficiale di Unity per il Machine Learning, scartata a causa della possibilità di usare esclusivamente il **Reinforcement Learning**;
- **ML.NET:** Libreria Microsoft per il machine learning, scartata dato che non era compatibile con la versione del framework .net utilizzato da Unity;
- **Python Scikit-Learn:** Libreria Python dedicata al Machine Learning, contenente diversi tool adatti ad una vasta gamma di situazioni;

La libreria selezionata è quindi **Scikit-Learn**.

E' stato dunque necessario creare un wrapper tra Python e C#.

Questo wrapper consiste nello stampare su file i “*weights*” ed i “*bias*” della rete neurale, così da poter effettuare predizioni all'esterno dell'ambiente Python.

Datasets & Learning

- Sono stati registrati diversi dataset, da 1 e da 2 *ottave*.
 - Ad ogni nota è associata una posizione delle mani, rappresentata da 500 righe contenenti dati riguardo le informazioni **FF**, **NFA** ed un identificativo numerico;
- Sono stati effettuati dei test utilizzando quantità diverse di cores della cpu:
 - 5 core: opzione adatta a macchine di fascia media, tempo impiegato circa 4 ore;
 - 20 core: opzione limitata a CPU con 24 core o più, tempo impiegato circa 1 ora;
- È stato usato lo scaling dei dati, ottenendo guadagni di accuratezza per il riconoscimento delle *gestures*;

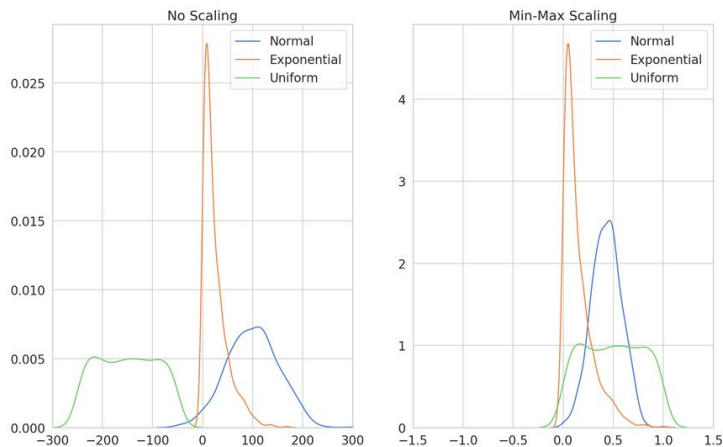
Scaling dei Dati

Le funzioni di scaling principali sono 3:

- Standardizzazione Z-Score;
- Normalizzazione MinMax Scaling;
- Normalizzazione Media;

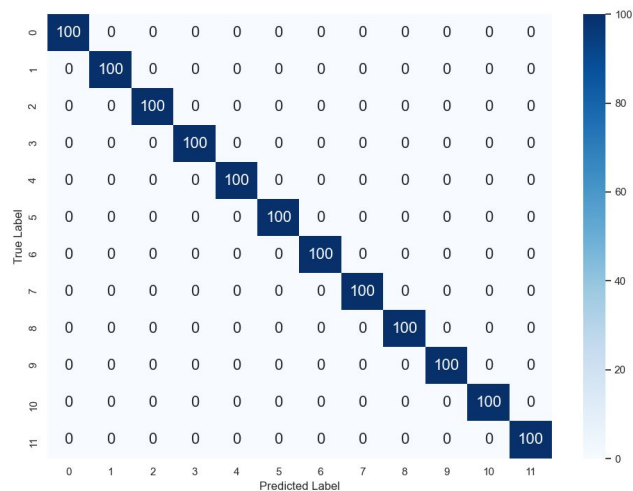
Ognuna di queste serve a portare i valori delle features entro un range, di solito tra 0 e 1.

La funzione di scaling da noi utilizzata è la **Normalizzazione MinMax Scaling**.

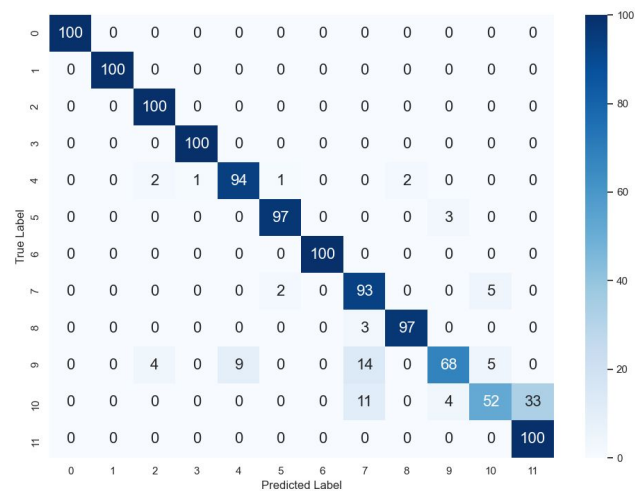


Test sulle Accuracy ed Overfitting

1 ottava, senza scaling



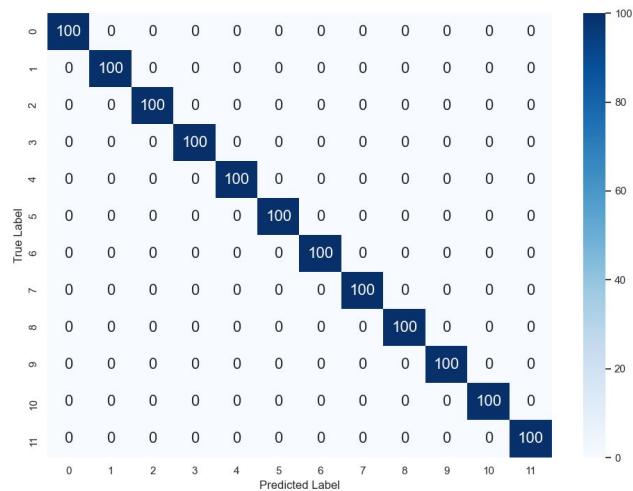
Accuracy stimata: 100%



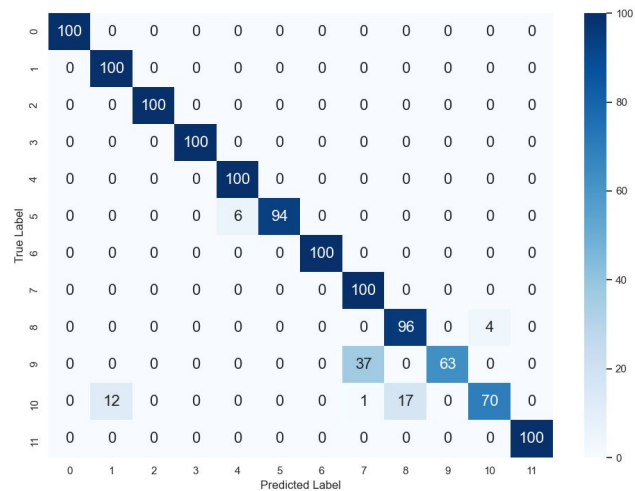
Accuracy in the wild: 91.75%

Test sulle Accuracy

1 ottava, con scaling



Accuracy stimata: 100%

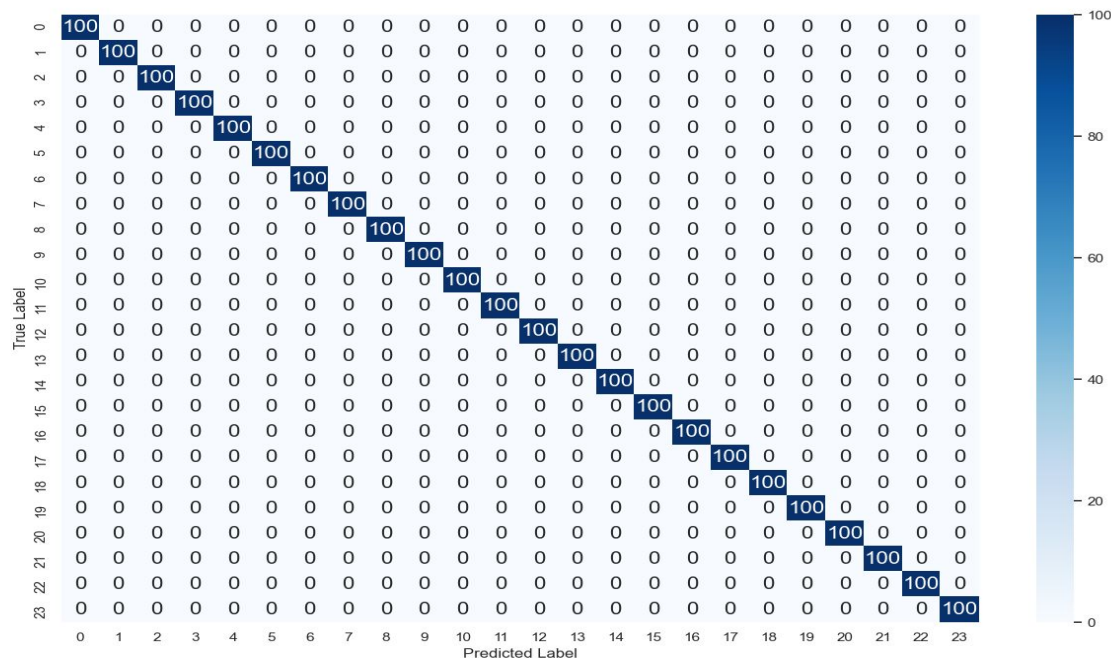


Accuracy in the wild: 94%

Test sulle Accuracy

2 ottave, senza scaling

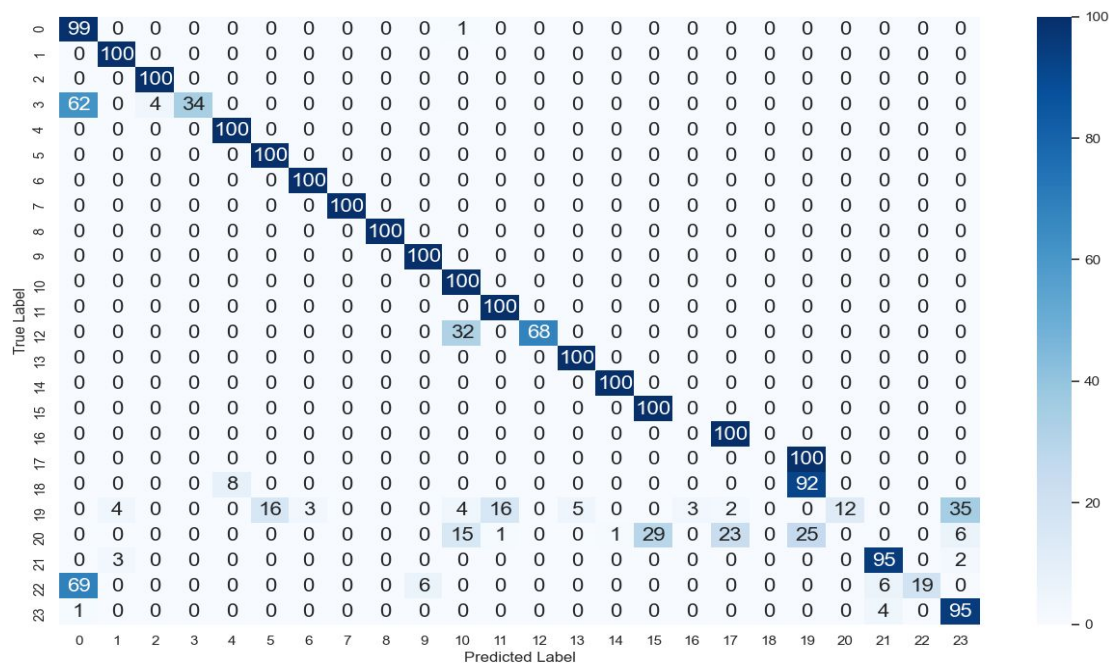
Accuracy stimata: 100%



Test sulle Accuracy

2 ottave, senza scaling

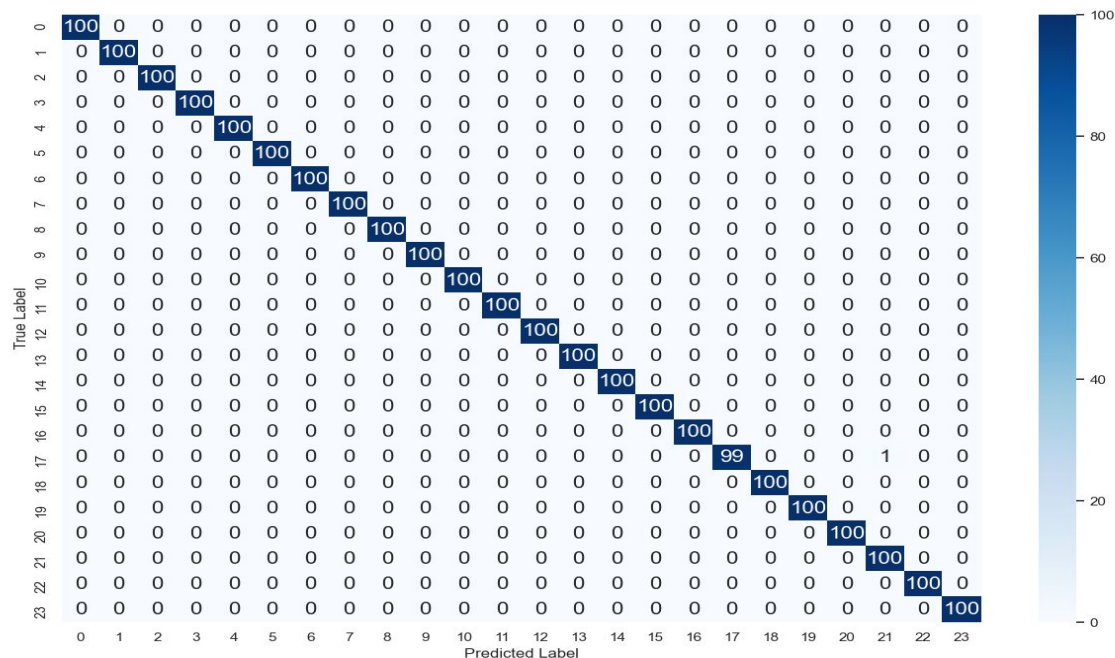
Accuracy in the wild: 71.30%



Test sulle Accuracy

2 ottave, con scaling

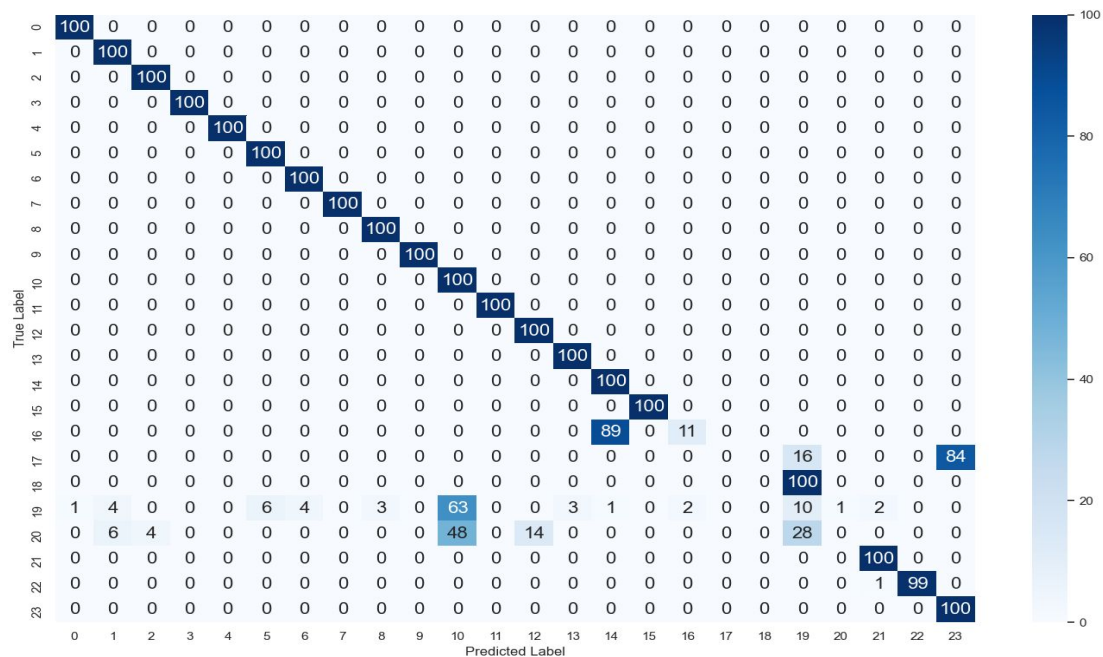
Accuracy stimata: 99.95%



Test sulle Accuracy

2 ottave, con scaling

Accuracy in the wild: 80%



Conclusione

Sono stati pensati alcuni lavori futuri per il progetto:

- Apprendimento tramite servizi Cloud;
- Incremento del volume del dataset;
- Utilizzo di più features e sensori;



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA



Grazie per l'attenzione.

Skibidi



UNIVERSITÀ DEGLI STUDI
DI SALERNO